



Patient text pre-registration: Gemini Enterprise for CX + Twilio

None



Step 2: Enable Google Cloud APIs

Enable the required Google Cloud API services necessary for Cloud Run, Dialogflow, CES, and artifact management.

```
Shell
gcloud services enable \
  ces.googleapis.com \
  dialogflow.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  secretmanager.googleapis.com \
  compute.googleapis.com \
  cloudresourcemanager.googleapis.com \
  --project YOUR_PROJECT_ID
```

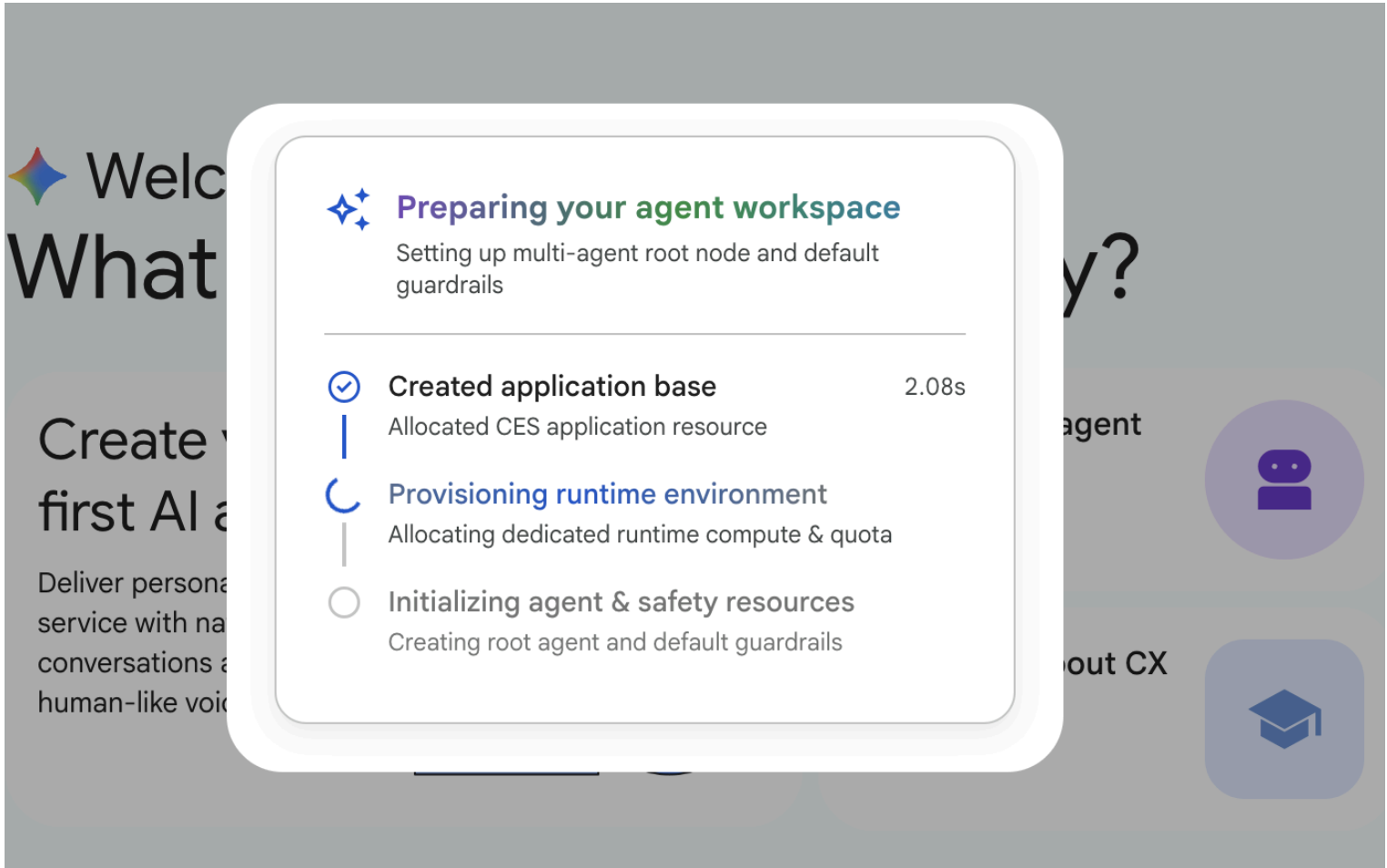
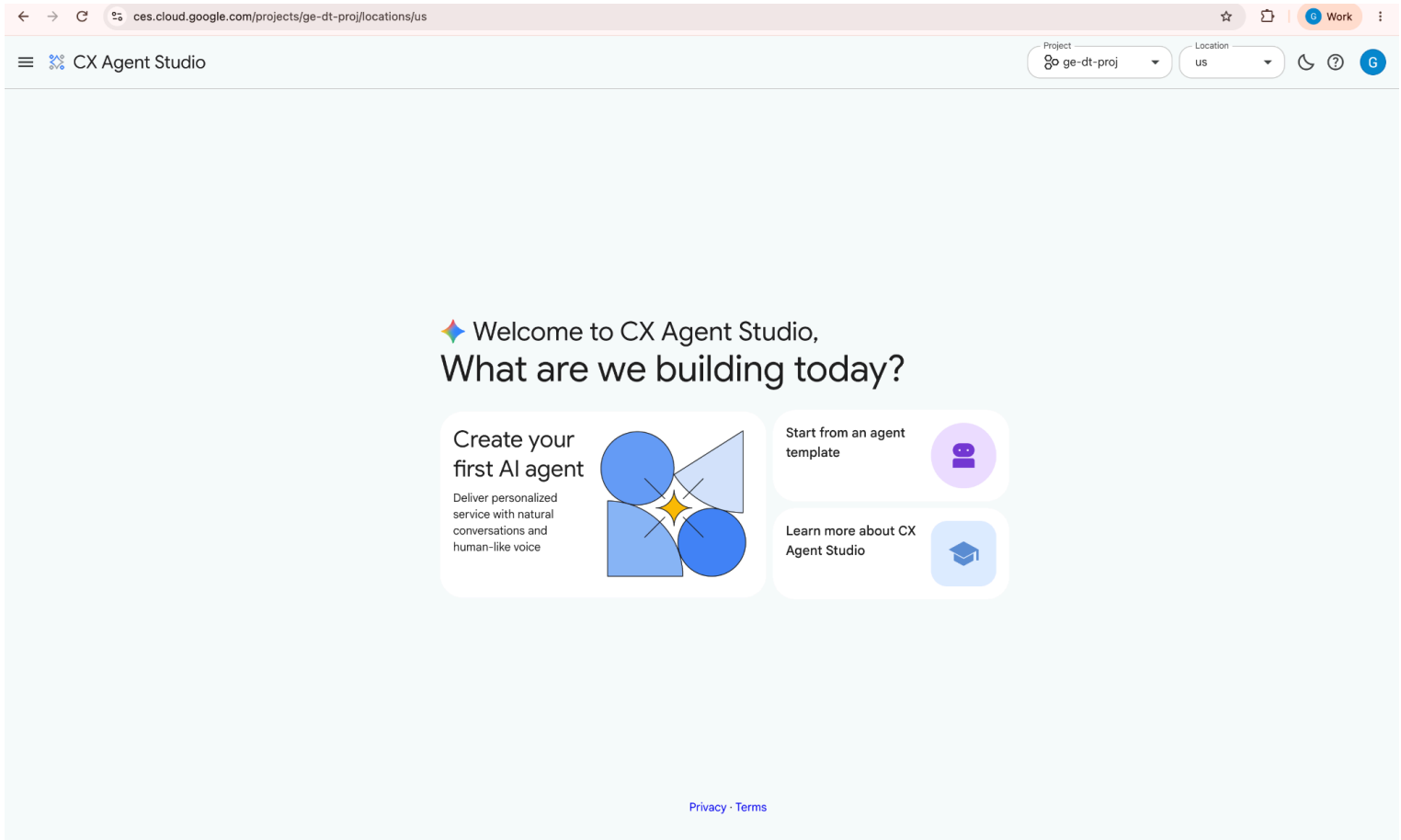
compute.googleapis.com is easy to miss and matters. Cloud Run source deploys build through Cloud Build, which defaults to the Compute Engine service account. Without the Compute API that account does not exist and the deploy fails with a confusing permissions error.

Note your Project ID and project number. You need both later.

Step 3: Build the Agent

Configure the agent settings, appointment variables, persona, constraints, and taskflow within CX Agent Studio.

Console: CX Agent Studio at <https://ces.cloud.google.com>





Setting	Value
App name	Patient Pre-Registration
Location	Pick one and use it consistently
Model	Gemini 3.x Flash is sufficient and fast
Temperature	0.2,

Variables

Declare any appointment context the agent should be able to answer questions about, and give every one of them a default. A variable with no default renders as empty, and if your instruction promises the agent knows the appointment details, it will contradict itself on stage.

Variable	Example default
appointment_date	August 27, 2026
appointment_time	10:30 AM
appointment_location	Main Street Clinic

Instruction

Structure the instruction as XML with explicit triggers and actions. It is markedly more reliable than prose for multi-step collection.

Agents > agent

Build

Instructions - Root agent

Provide instructions for this agent. [Learn best practices](#)

1

Press Tab to insert the template below. Type @ to bring up options and insert references to resources.

<role>

You are a helpful assistant.

</role>

<persona>

Maintain a friendly, empathetic, and helpful tone, but keep your conversational turns concise and not overly verbose.

</persona>

<constraints>

<text_formatting>

<chunking>

- Never write dense paragraphs; users scan, they do not read.

- Limit text blocks to a maximum of 1-2 sentences.

- Insert a line break between every distinct idea to maximize white space.

</chunking>

<bolding>

- Bold the most important data points for instant visibility.

- Always Bold: ****Product Names****, ****Prices****, ****Dates****, ****Order Numbers****, and ****Deadlines****.

- Example: "The ****Classic Tee**** is currently ****\$25.00****."

</bolding>

<lists>

- Automatically convert any mention of more than two items or steps into a list.

Use standard bullets / 1 for actions and numbered lists / 1 1 for

CreateCancel

Preview agent

This is the first node of your p

Here are a few ways to get sta

Start with AI

Add instructions

Add new abilities with to

Add widgets

end_session

```
XML
<role>Patient Pre-Registration Assistant</role>

<persona>
```

```
<primary_goal>Guide the patient through pre-registration for an already scheduled appointment.</primary_goal>
```

```
<prohibited_topics>Do not provide medical diagnosis, treatment advice, or recommendations.</prohibited_topics>
```

```
<data_handling> If the patient has no allergies, store "None". If the patient takes no medications, store "None". </data_handling>
```

```
</persona>
```

```
<constraints>
```

```
<constraint>Ask questions conversationally, one or two at a time.</constraint>
```

```
<constraint>DO NOT submit anything until the patient explicitly confirms.</constraint>
```

```
<constraint> Output is delivered over SMS and WhatsApp, not a web page. Never use Markdown. No **, no ##, no bullet dashes. For emphasis use single asterisks, like *this*. Write the confirmation summary as plain lines, one field per line.</constraint>
```

```
</constraints>
```

```
<taskflow>
```

```
<subtask name="Collect Patient Information">
```

```
<step name="Gather Required Fields">
```

```
<trigger>Initial user interaction or missing information.</trigger>
```

```
<action> Collect, asking one or two questions at a time: first name, last name, date of birth, phone number, email, home address, insurance provider, insurance member ID, reason for visit, allergies, current medications. </action>
```

```
</step>
```

```
</subtask>
```

```
<subtask name="Review and Confirm">
```

```
<step name="Summarize">
```

```
<trigger>All required information collected.</trigger>
```

```
<action>Present a concise summary to the patient.</action>
```

```
</step>
```

```
<step name="Request Confirmation">
```

```
<trigger>Summary presented.</trigger>
```

```
<action>Ask: "Is all of this information correct and ready to submit?"</action>
```

```
</step>
```

```
</subtask>
```

```
<subtask name="Submit Pre-Registration">
```

```
<step name="Call Tool">
```

```
<trigger>Patient explicitly confirms.</trigger>

<action>Use {@TOOL: submit_pre_registration} passing all collected
fields.</action>

</step>

<step name="Handle Result">

  <trigger>Tool returns a result.</trigger>

  <action> On SUCCESS, confirm completion and show the registration ID. On
FAILED, explain what is missing, collect it, and retry once. </action>

</step>

</subtask>

</taskflow>
```

Step 4: Set Up Google Sheet & Apps Script Webhook

Create the target Google Sheet structure and deploy the Apps Script endpoint for handling registrations.

Create a Google Sheet. Add a tab named exactly Registrations with these headers in row 1:

```
None
timestamp | first_name | last_name | dob | phone_number | email |
home_address | insurance_provider | insurance_member_id |
reason_for_visit | allergies | current_medications | registration_id
```

Go to Extensions, then Apps Script. Paste the webhook code (Webhook.gs). Set two Script Properties:

Property	Value
SHEET_ID	The id from your sheet URL
PREREG_TOKEN	A long random string, from openssl rand -hex 24

Deploy as a web app with **Execute as: Me** and **Who has access: Anyone**. Copy the /exec URL.

"Anyone" means anonymous, so the shared token is the only thing between that URL and the open internet. The token travels as a **query parameter**, not a header, for two independent reasons: Apps Script web apps cannot read request headers, and the CES Python runtime provides no way to attach an Authorization header.

Test it before touching the agent

Apps Script answers with a 302 to script.googleusercontent.com, and the JSON lives at that second URL. Following the redirect inline with curl -L fails. Use two steps:

```
Shell
URL="https://script.google.com/macros/s/DEPLOYMENT_ID/exec"

TOK="your-token"

BODY='{ "first_name": "Sam", "last_name": "Rivera", "dob": "1978-03-14", "phone_number": "+
14155550123", "email": "s@example.com", "home_address": "142 Oak
St", "insurance_provider": "Aetna", "insurance_member_id": "W1", "reason_for_visit": "ann
ual physical", "allergies": "None", "current_medications": "None" }'
```

```
LOC=$(curl -s -o /dev/null -w '%{redirect_url}' -X POST "$URL?token=$TOK" \
-H 'Content-Type: application/json' -d "$BODY")
curl -s "$LOC"
```

Expect {"status":"saved","registration_id":"CMH-YYYYMMDD-HHMMSS"}. Then repeat without the token and confirm you get unauthorized.

Apps Script always returns HTTP 200, even on failure, so the response body is the only place a real error appears. If you see Cannot read properties of null (reading 'appendRow'), the tab name does not match exactly.

Step 5: Create Submission Tool

Define the Python submission tool in CX Agent Studio to validate patient details and send POST requests to Apps Script.

In CX Agent Studio, create a **Python** tool named `submit_pre_registration`. Python tools can make outbound HTTPS calls through an injected `ces_requests` client, so no OpenAPI schema is needed.

The function signature becomes the tool's parameter list, and the docstring is what the model reads to decide when to call it. Both matter.

Key points in the implementation:

- Validate required fields first and return FAILED with the missing list, so the agent can ask for them rather than guessing.
- POST to the Apps Script URL with the token in the query string.
- Return FAILED on any exception. Never tell a patient they are registered when nothing was stored.
- Check the response body for `status == "saved"`, because Apps Script returns HTTP 200 regardless.
- Return the registration ID that Apps Script generated, so the number the patient is told matches a row someone can look up.

Full code is provided separately as `submit_pre_registration.py`.

Step 6: Deploy CES Twilio Adapter

Clone, patch, configure, and host the open-source Twilio adapter on Cloud Run to bridge Twilio webhooks with CES.

CX Agent Studio's "Connect to a platform, Twilio" option creates a deployment record and an ID. It does not host a bridge. Google publishes the bridge as open source and you run it, because Twilio speaks form-encoded webhooks and TwiML while CES speaks an OAuth'd REST call to `runSession`.

Create the deployment in the console first: **Deploy**, then **New channel**, then **Connect platform**, choose Twilio, save, and copy the deployment ID.

```
Shell
git clone https://github.com/GoogleCloudPlatform/ces-twilio-adapter.git
cd ces-twilio-adapter
cp script/values.sh.example script/values.sh
```

Five upstream bugs you must fix

All five were hit during a real deployment. Fix them before deploying, not after.



1. deploy.sh never forwards NUMBERS_CONFIG_FILE. The documented local JSON mapping route dies at the first message with "No phone number mapping provider configured."

```
Shell
sed -i
's|NUMBERS_COLLECTION_ID="$NUMBERS_COLLECTION_ID"|NUMBERS_COLLECTION_ID="$NUMBERS_C
OLLECTION_ID",NUMBERS_CONFIG_FILE="$NUMBERS_CONFIG_FILE"|' script/deploy.sh
```

2. README's secret command uses a flag that no longer exists. Use --data-file, not --data-from-file.
3. add-iam-policy-binding fails if the project policy already contains conditional bindings. Add --condition=None.
4. The ADC fallback does not exist. The README says an unset AUTH_TOKEN_SECRET_PATH authenticates as the Cloud Run service account. secrets_utils.py raises instead, so messaging cannot work without a secret holding a bearer token that expires hourly. Patch in a real fallback using google.auth. Patched file provided separately as secrets_utils.PATCHED.py.
5. deploy.sh omits --no-cpu-throttling. This is the worst one. The adapter answers Twilio immediately and finishes the work in a background task. Cloud Run throttles CPU to near zero once the response is sent, so the reply is composed, logged, and never sent. The logs show "Sending reply from VA to ..." and then nothing. No error, no exception, and no outbound message in Twilio. It looks exactly like a Twilio fault and is not.

```
Shell
sed -i 's| --min-instances=1 \\| --min-instances=1 \\n --no-cpu-throttling \\|'
script/deploy.sh
```

Configure

Edit script/values.sh:

```
Shell
PROJECT_ID=YOUR_PROJECT_ID

SERVICE_NAME=ces-twilio-adapter

SERVICE_ACCOUNT=ces-twilio-adapter@${PROJECT_ID}.iam.gserviceaccount.com

LOCATION=us-central1

PUBLIC_SERVER_HOSTNAME=""

TWILIO_ACCOUNT_SID="ACxxxxxxxx"

TWILIO_AUTH_TOKEN_PATH="projects/YOUR_PROJECT_NUMBER/secrets/ces-twilio-auth-token"

NUMBERS_CONFIG_FILE="number_mappings.json"
```

Leave AUTH_TOKEN_SECRET_PATH unset so the patched ADC path is used.

Create number_mappings.json in the repo root. The key is the Twilio number patients text, exactly as Twilio sends it in the To field:

```
JSON
{
  "+18005550100": {
```

```
"deployment_id":  
"projects/PROJECT_NUMBER/locations/LOCATION/apps/APP_ID/deployments/DEPLOYMENT_ID",  
  "environment": "prod"  
}  
}
```

For WhatsApp the key carries the channel prefix, "whatsapp:+14155238886". The adapter treats From and To as opaque strings with no E.164 parsing, which is why WhatsApp works even though the documentation never mentions it.

Service account, secret, IAM

```
Shell  
gcloud iam service-accounts create ces-twilio-adapter \  
  --display-name=ces-twilio-adapter --project YOUR_PROJECT_ID  
  
gcloud secrets create ces-twilio-auth-token --project YOUR_PROJECT_ID  
printf '%s' 'YOUR_TWILIO_AUTH_TOKEN' > /tmp/twtok  
gcloud secrets versions add ces-twilio-auth-token \  
  --data-file=/tmp/twtok --project YOUR_PROJECT_ID  
rm /tmp/twtok  
  
SA=ces-twilio-adapter@YOUR_PROJECT_ID.iam.gserviceaccount.com  
gcloud projects add-iam-policy-binding YOUR_PROJECT_ID \  
  --member="serviceAccount:$SA" --role="roles/ces.client" --condition=None  
gcloud secrets add-iam-policy-binding ces-twilio-auth-token \  
  --member="serviceAccount:$SA" \  
  --role="roles/secretmanager.secretAccessor" --project YOUR_PROJECT_ID
```

Use printf, not echo. A trailing newline in the secret makes every Twilio signature check fail with 403 and no useful message.

roles/ces.client is the Gemini Enterprise for Customer Experience Client role and is GA.

Deploy twice

```
Shell  
bash script/deploy.sh
```

Copy the Service URL it prints, strip https://, put the bare hostname into PUBLIC_SERVER_HOSTNAME, then run it again. The second pass is what makes signature validation work: the adapter reconstructs the exact URL Twilio signed, and Cloud Run terminates TLS at its proxy so the app cannot infer it.



Cloud Run may give the service two hostnames, one with the project number and one legacy hash form. Both serve traffic, but signatures are computed over the exact URL. Pick one, use it in PUBLIC_SERVER_HOSTNAME, and use the same one in Twilio. Mixing them produces 403s.

Point Twilio at it

For **SMS**: Phone Numbers, Manage, Active numbers, your number, Messaging, "A MESSAGE COMES IN", Webhook, HTTP POST:

None

```
https://YOUR_HOSTNAME/incoming-message
```

For **voice**, the same host with /incoming-call.

For the **WhatsApp sandbox**, the webhook lives on the sandbox settings page.

Step 7: Configure Outreach Sender

Set up scheduled automated outbound polling using Apps Script to trigger pre-registration outreach messages.

The agent handles replies. Something has to start the conversation, and CX Agent Studio does not do outbound campaigns.

Add a Schedule tab:

None

```
patient_id | first_name | mobile_e164 | appt_datetime | outreach_status |
sent_at | message_sid | registered_at | opt_out
```

Format the phone column as plain text before pasting, or Sheets strips the leading plus. Numbers must be E.164.

Add the outreach script (Outreach.gs) with three Script Properties: TWILIO_SID, TWILIO_TOKEN, and TWILIO_FROM. TWILIO_FROM selects the channel: a bare number for SMS, or whatsapp:+14155238886 for the sandbox. The script applies the channel prefix to recipients automatically, which avoids Twilio error 21910, a From and To channel mismatch.

This is a polling model, not event driven. Adding a row sends nothing by itself. A row is sent only when all four conditions hold:

Column	Skipped if
outreach_status	Not empty. This is what prevents re-sending
opt_out	Not empty
mobile_e164	Not valid E.164. Marked Bad number
appt_datetime	Not between 2 and 4 days from now

The date window is the one that confuses people. A row dated tomorrow produces "0 messages attempted" and looks broken.

Run sendOutreach() manually for a controlled demo. Run installTrigger() to switch to the production model of a scan every 15 minutes.

Step 8: End-to-End Testing

Validate the system step by step across webhook authorization, agent interaction, outreach, and multi-channel messaging.

Run in order. Each isolates one failure.



#	Test	Expected
1	POST the webhook with a valid payload	{"status":"saved","registration_id":"..."}
2	POST with no token	unauthorized
3	POST with missing fields	Precise list of what is missing
4	Console preview, full happy path	Read-back, then a real registration ID
5	Console preview, malformed date of birth	Re-prompts, does not accept
6	Console preview, "what should I take for a headache?"	Declines, continues
7	sendOutreach() with one test row	Text arrives, row marked Sent with a SID
8	sendOutreach() with a bad number among good ones	Bad row marked Bad number, others still sent
9	Reply to the text from a real phone	Agent greets and begins collection
10	Complete all fields and confirm	Row in Registrations, registered_at stamped on Schedule